

VIETNAM NATIONAL UNIVERSITY, HANOI
UNIVERSITY OF ENGINEERING AND TECHNOLOGY



Bui Minh Thang

**DESIGN AND EVALUATION OF A CONTEXT-AWARE
RESOURCE PROFILE SELECTION EXTENSION FOR
JUPYTERHUB ON KUBERNETES**

GRADUATION THESIS

Major: Computer Networks and Data Communications

HA NOI – 2026

VIETNAM NATIONAL UNIVERSITY, HANOI
UNIVERSITY OF ENGINEERING AND TECHNOLOGY

Bui Minh Thang

**DESIGN AND EVALUATION OF A CONTEXT-AWARE
RESOURCE PROFILE SELECTION EXTENSION FOR
JUPYTERHUB ON KUBERNETES**

GRADUATION THESIS

Major: Computer Networks and Data Communications

Supervisor: Dr. Hoang Xuan Tung

HA NOI – 2026

ABSTRACT

Shared JupyterHub deployments commonly expose administrator-defined notebook environments through manual profile and image selectors. This preserves operational control, but requires users to translate a workload described in semantic terms into software-image and Kubernetes resource choices before a pod exists. This thesis studies Intent-Spawner, a bounded pre-spawn decision-support workflow for that choice. Its principal method, P2, converts natural-language workload intent and explicit hints into a validated structured representation, retrieves administrator-curated candidates using BM25 and dense retrieval, fuses their ranks with Reciprocal Rank Fusion, removes candidates that violate hard constraints, and ranks feasible candidates deterministically. The selected identifier is resolved from trusted administrator data and checked against policy, profile allowlists, catalog versions, and digest-pinned image identity. A preview lets the authenticated user confirm, edit, or choose an allowlisted manual override. A single-use, user-bound token binds the confirmed preview to the pre-spawn hook, which applies the stored decision without recomputing the recommendation.

The thesis defines B0 as ordinary manual JupyterHub selection, P1 as a frozen rule-based recommender, P2 as the proposed structured-intent hybrid-retrieval method, and P3 as an optional grounded language-model reranker. Protocol-v5 separates offline recommendation and language robustness (E1/E2), human outcomes (E3), independently observed resource envelopes (E4), and image functionality and storage (E5). P3 was not retained after its development gate. Available evidence includes a development-only E1/E2 run over 18 workload families and 83 prompt variants, local OrbStack E4 observations of 421 Pod trials across 20 workload families, and E5 Docker/OCI observations for four digest-pinned images on linux/arm64. These packages are bounded observations, not confirmatory evaluations. E3 has no participant observations; the consolidated Protocol-v5 analysis is INCOMPLETE and all confirmatory decisions are NOT_EXECUTED. The thesis therefore presents the system design, implementation, and evaluation infrastructure, but does not claim confirmatory superiority, user benefit, general cluster efficiency, production readiness, or realized node-storage savings.

The main contribution is an auditable composition of workload-intent parsing, hybrid retrieval, deterministic feasibility and ranking, trusted catalog resolution, and explicit human confirmation at the JupyterHub pre-spawn boundary. The work also establishes a reproducible protocol

for evaluating recommendation quality, language robustness, human outcomes, resource envelopes, and image behavior as distinct constructs. Optional dynamic sizing and stop/recreate reprovisioning are prototype extensions; neither is an exact demand oracle nor live migration.

Keywords: JupyterHub; Kubernetes; workload intent; hybrid retrieval.

AUTHORSHIP DECLARATION

I declare that this thesis is my own work, except where the contributions of others are explicitly acknowledged and cited. I have not presented planned or incomplete experiments as completed results. All empirical statements in this manuscript are limited to the provenance and evidence status recorded in the repository.

Signature: _____

ACKNOWLEDGEMENTS

The author would like to express sincere gratitude to the thesis supervisor, Dr. Hoang Xuan Tung, for guidance, critical feedback, and encouragement throughout the development and evaluation of this work. The author is also grateful to the lecturers and staff of the Faculty of Information Technology, University of Engineering and Technology, Vietnam National University, Hanoi, for the technical and academic foundation that made this project possible.

The implementation and evaluation work required repeated review of JupyterHub, Kubernetes, information-retrieval, recommender-system, container-image, and human-computer-interaction concepts. Appreciation is extended to colleagues, friends, and family members whose discussions and practical support helped sustain this iterative process. Any remaining errors, limitations, or interpretation choices in this thesis are the responsibility of the author.

Contents

Abstract	i
Authorship Declaration	ii
Acknowledgements	iii
List of Figures	viii
List of Tables	ix
Abbreviations	x
Chapter 1. Introduction	1
1.1. Shared notebook infrastructure and the configuration decision	1
1.1.1. Problem statement	2
1.2. Research questions	3
1.2.1. Objectives and contributions	4
1.3. Scope and assumptions	4
1.3.1. Methodological stance and expected use	5
1.4. Thesis organization	6
Chapter 2. Background and Related Work	7
2.1. JupyterHub, KubeSpawner, and curated environments	7
2.1.1. Kubernetes resource semantics and feasibility	7
2.1.2. Resource estimation and workload-aware management	8
2.2. Intent and natural-language configuration	9
2.2.1. Sparse, dense, and fused retrieval	10
2.2.2. Constraint-aware recommendation and decision support	11
2.3. Human-centered evaluation	12

2.3.1. Container images, layers, and storage accounting	12
2.3.2. Evaluation concepts and statistical discipline	13
2.4. Catalog quality and metadata	14
2.4.1. Operational reliability and user trust.....	15
2.4.2. Design trade-offs in a curated environment catalog.....	16
2.4.3. Terminology and construct boundaries.....	17
2.4.4. Research gap and synthesis	18
Chapter 3. Proposed Method	19
3.1. System taxonomy and design objective	19
3.1.1. Structured intent representation	19
3.1.2. Hybrid candidate retrieval	20
3.2. Hard constraints and deterministic ranking	21
3.2.1. Trusted candidate corpus and policy boundary.....	22
3.2.2. Preview, confirmation, and pre-spawn binding	23
3.3. Rationale for the selected architecture.....	23
3.3.1. Optional dynamic sizing and reprovisioning.....	24
3.3.2. Ranking pipeline and score interpretation	25
3.4. Algorithmic outline	26
3.4.1. Threat model.....	26
3.4.2. Privacy, auditability, and reproducibility	27
Chapter 4. Implementation and Experiments.....	29
4.1. Implemented workflow and evidence design	29
4.1.1. Repository implementation	29
4.1.2. Failure handling and safety properties.....	30

4.1.3. Protocol-v5 scope and empirical status	30
4.1.4. Reproducibility and interpretation rules	32
4.2. Offline recommendation and robustness results	32
4.2.1. E1: family-level recommendation quality	32
4.2.2. E2: language and prompt robustness.....	34
4.2.3. E5 recommendation metrics and cross-report reconciliation	34
4.3. Cluster and image observations	35
4.3.1. E4: observed local resource trials.....	35
4.3.2. E5: functional capability probes	37
4.3.3. E5: OCI layer accounting and catalog scale	38
4.4. Synthesis, limits, and implications	39
4.4.1. E3 and human outcomes	40
4.4.2. P3 development gate and formal claim registry	40
4.4.3. What the evidence supports	41
Chapter 5. Conclusion	43
5.1. Conclusion	43
5.2. Limitations	44
5.2.1. Implications for administrators and users.....	44
5.3. Prioritized future work	45
5.3.1. Final thesis position	46
References	46

List of Figures

3.1. P2 transforms a bounded intent into a confirmed catalog choice before the
ordinary Kubernetes spawn path.20

4.1. Locally observed memory envelopes and static profiles in the E4 package36

4.2. Logical and unique compressed OCI layer-byte totals for the observed four-
image catalog prefix.....39

List of Tables

4.1. Protocol-v5 experiment streams and evidence scope 31

4.2. Observed E1 development estimates for P1 and P2 33

4.3. Observed E2 robustness metrics on the development split..... 34

4.4. E5 observed label, catalog, and functional dimensions 37

4.5. Observed four-image compressed OCI layer-byte accounting 38

4.6. Evidence boundary for current thesis claims 42

Abbreviations

API	Application Programming Interface
BM25	Best Matching 25
CPU	Central Processing Unit
GPU	Graphics Processing Unit
HCI	Human-Computer Interaction
IR	Information Retrieval
LLM	Large Language Model
MRR	Mean Reciprocal Rank
nDCG	Normalized Discounted Cumulative Gain
OCI	Open Container Initiative
POD	Plain Old Data / Kubernetes Pod (context dependent)
RRF	Reciprocal Rank Fusion
RQ	Research Question
TTL	Time to Live
VPA	Vertical Pod Autoscaler

Chapter 1.

Introduction

1.1. Shared notebook infrastructure and the configuration decision

Computational notebooks combine executable code, computed results, and explanatory prose in an interactive document. They support exploratory data analysis, teaching, scientific computing, and machine-learning development because a user can move between an idea and its execution without first building a complete application. A notebook service used by a class, laboratory, or organization must also manage identity, isolate sessions, provide storage, and offer environments that remain maintainable over time. JupyterHub provides the multi-user control plane for these services, while a Spawner implementation creates each user's server. In a Kubernetes deployment, KubeSpawner maps a user's server to a Pod and associated resources. Administrators commonly expose a finite set of profiles and container images through configuration such as a profile list [17, 30, 15, 38].

This curated-choice model is operationally important. An administrator can build known images, constrain CPU and memory, decide which accelerators may be requested, and avoid letting each user provide an arbitrary image reference. The same design creates a decision burden at the interface. Users think in terms of work: load a dataset, preprocess text, fit a model, or process images. The platform asks them to select an environment, image, and sometimes resource profile. A person who knows the task but not the catalog internals may choose a profile by name, choose a familiar framework image that is heavier than needed, or select an environment that lacks a required capability.

Intent-Spawner investigates a bounded decision-support layer at this boundary. Its purpose is to map a description of the current task and any explicit constraints to a ranked set of administrator-approved alternatives. It does not replace Kubernetes scheduling, resource admission, quotas, or runtime monitoring. It does not promise to know a workload’s peak demand before execution. It seeks to make the profile-selection decision more informed, reviewable, and reproducible while preserving user choice and platform policy.

1.1.1. Problem statement

Let a user request be a description q of an intended computation, possibly accompanied by explicit statements such as a required library, GPU requirement, minimum memory, or forbidden capability. Let C be a finite catalog of administrator-curated profile-image candidates. Each candidate contains descriptive metadata, supported capabilities, resource characteristics, and a reference to an approved image. The system must present relevant candidates without treating semantic similarity as proof of feasibility.

This distinction is central. A candidate can mention a desired framework while failing a hard memory bound. Another candidate can be feasible but rank poorly because a task is described using a paraphrase rather than the catalog’s keywords. An unconstrained language model could generate plausible configuration that is not approved, while a fixed rule list can fail on natural variation. A useful recommender therefore needs to separate four operations: interpret intent into structured data; retrieve likely candidates; deterministically enforce hard requirements and policy; and allow the user to inspect and confirm the choice. The final Pod still passes through Kubernetes admission and scheduling, which remain authoritative for cluster state [21, 19, 20, 23, 22].

The research problem is consequently not general cloud resource optimization. It is a pre-spawn selection problem in a controlled catalog. The proposal is judged across separate dimensions: offline recommendation quality, stability under language variation, human choice and effort, resource-envelope adequacy, and image functionality and storage accounting. Evidence from one dimension cannot substitute for another. For example,

an offline label match does not establish that a participant made a better decision, and a container-layer accounting result does not establish savings on a particular cluster’s node filesystem.

1.2. Research questions

The thesis follows six research questions defined by the Protocol-v5 experiment plan. They are deliberately framed so that empirical evidence can be collected and interpreted independently.

1. **RQ1: Recommendation quality.** Under a frozen candidate catalog and development-independent evaluation split, how does P2 compare with the frozen P1 recommender on workload-family-level candidate quality?
2. **RQ2: Language robustness.** How does P2 behave when the same workload family is expressed through controlled paraphrases, abbreviations, and language forms?
3. **RQ3: Human outcomes.** In a within-participant task, does P2 change completion time, choice quality, confidence, perceived workload, or usability relative to ordinary manual profile selection (B0)?
4. **RQ4: Resource envelopes.** For a declared set of workload families and an eligible Kubernetes environment, how do independently measured safe resource envelopes compare with available static profiles and policy-bounded recommendations?
5. **RQ5: Image functionality and storage.** Do approved images pass bounded functional probes, and how does content-addressed layer reuse affect logical versus unique layer-byte accounting for the measured catalog?
6. **RQ6: Optional LLM reranking.** Does a grounded reranking branch meet its predeclared development gate, and, if retained, does it provide enough benefit to justify its added cost and complexity?

These questions do not imply that every hypothesis has been decided. The current consolidated research-analysis package reports INCOMPLETE, and registered

confirmatory decisions remain NOT_EXECUTED. RQ1 and RQ2 have development observations, RQ3 lacks participant observations, and E4/E5 packages are bounded local or development observations. The thesis treats these statuses as part of the result rather than as missing text to be filled by assumption.

1.2.1. Objectives and contributions

The objectives are to: (1) define a safe and auditable pre-spawn recommendation workflow; (2) implement structured intent, hybrid candidate retrieval, constraints, deterministic ranking, trusted catalog resolution, preview, confirmation, and hook integration; (3) define separate experiment streams and provenance requirements; and (4) report the currently available evidence without overstating its scope.

The principal contribution is a systems integration of established techniques at a practical JupyterHub boundary. The work does not claim to invent BM25, dense retrieval, rank fusion, constraint checking, or container-layer deduplication. It composes them into a workflow in which untrusted request text cannot directly become arbitrary Kubernetes configuration. A second contribution is the explicit boundary between a development decision-support feature and the platform’s authoritative controls. A third is an evidence-oriented evaluation design that distinguishes semantic workload families from prompt variants and repeated executions. Finally, the repository contains a working prototype and machine-readable experiment harnesses that make later confirmation possible.

1.3. Scope and assumptions

The target setting is a JupyterHub installation in which an administrator owns the candidate catalog and the user is authenticated before requesting a preview. The principal method is P2. B0 is the ordinary manual profile/image selection workflow and generates no ranking. P1 is the existing frozen rule-based comparator. P3 is an optional grounded language-model reranker which was not retained after its development gate. The main

text concerns the recommendation service and its interface to the pre-spawn configuration path; it is not an evaluation of the scheduler, node autoscaler, or all of JupyterHub.

The term *context-aware* refers to the current task description, explicit structured requirements, and the current administrator-owned catalog and policy. The implementation is not a longitudinal personalization system. It does not infer user skill from history or inspect notebook contents to rank environments. It does not observe live cluster headroom during P2 candidate ranking. User history, when considered as future work, would require a separate privacy and evaluation design.

Empirical scope is similarly bounded. E1/E2 are development evidence; E4 records local OrbStack Pod/workload observations; E5 records digest-pinned image probes and OCI manifest layer accounting for a small set of images on linux/arm64. E3 has no participants. The statistical-analysis package has audit issues and has not issued confirmatory decisions. Accordingly, this thesis does not claim statistically confirmed improvement over P1, improved user outcomes over B0, generalized resource efficiency, GPU execution, production readiness, or realized storage savings in a multi-node registry or cluster.

1.3.1. Methodological stance and expected use

The thesis adopts a design-and-evaluation stance. It first defines the system boundary and translates the user problem into explicit functional requirements. It then implements a prototype in the repository and evaluates components through separate evidence streams. This approach is suitable for a systems thesis because a recommender at a pre-spawn boundary must satisfy both software integration requirements and research evaluation requirements. A ranking-only prototype would leave unresolved the question of how the choice is applied, while an implementation-only demonstration would leave the quality and limitations of its recommendations unexamined.

The expected use is assistive. The system should reduce the effort required to understand a catalog, surface the alternatives that plausibly fit a task, and let a user make an informed selection. Administrators continue to define images, resource profiles, and

policy. The user remains responsible for confirming the option in the current prototype workflow. Platform controls remain responsible for final admission and scheduling. This allocation of authority is intentional: the recommender has incomplete information about runtime behavior and cluster state, while administrators and users possess different but complementary context.

Several design requirements follow. The method must produce a candidate drawn from trusted data; preserve explicit requirements through filtering; make its suggestion reviewable; handle infeasibility without silently weakening constraints; and reproduce the same order under the same versioned inputs. The evaluation must use comparisons that reflect the task, maintain an independent unit appropriate to each experiment, and report unexecuted conditions explicitly. These requirements connect the technical method in Chapter 3 to the experiment design and claim boundaries in Chapter 4.

1.4. Thesis organization

Chapter 2 introduces notebook infrastructure, Kubernetes resources, resource estimation, intent interpretation, retrieval, recommendation constraints, user-centered evaluation, container images, and related work. Chapter 3 presents the P2 method, its formal ranking problem, trusted-data boundaries, confirmation mechanism, and optional extensions. Chapter 4 describes the prototype implementation and Protocol-v5 evaluation design, then reports available evidence with its provenance and limitations. Chapter 5 discusses the implications and threats to validity, concludes the thesis, and sets out prioritized future work. The five-chapter organization follows the requested LaTeX chapter files; implementation and experiments are integrated into Chapter 4 so that the manuscript remains aligned with that structure.

Chapter 2.

Background and Related Work

2.1. JupyterHub, KubeSpawner, and curated environments

JupyterHub separates the multi-user service from the individual notebook server. The Hub authenticates users, routes them to their servers, and delegates server creation through a Spawner abstraction. KubeSpawner realizes that abstraction by creating Kubernetes resources for each user. Zero to JupyterHub (Z2JH) packages a common deployment model and supports administrator-defined profiles that map user choices to approved overrides [17, 30, 15, 38].

A profile can bundle a software image with resource requests, limits, storage choices, environment variables, and other server settings. This catalog acts as a policy boundary: the user selects among reviewed options, while the administrator remains responsible for what can be deployed. The default B0 workflow is intentionally retained as the baseline because it represents an existing and operationally useful user experience. Intent-Spawner does not assume that manual profiles are defective. It asks whether an optional translation and ranking step helps users navigate a catalog whose names and technical properties may not match their vocabulary.

The Spawner lifecycle also determines where the method belongs. The recommendation is made before a user's Pod is created, and a stored decision is applied during pre-spawn configuration. The method therefore cannot know the node that will eventually host the Pod or guarantee that the cluster will admit it. This is why Intent-Spawner should be described as pre-spawn decision support, not as a Kubernetes scheduler or autoscaler.

2.1.1. Kubernetes resource semantics and feasibility

Kubernetes resource requests and limits have distinct roles. Requests contribute to scheduling feasibility and resource accounting; limits bound container use. CPU limits can be enforced through throttling, while memory limit violations can result in termination. Pod QoS classification is also affected by resource configuration. GPUs and other accelerators are typically exposed as extended resources through device plugins, while ResourceQuota constrains aggregate namespace usage [21, 19, 20, 23, 22].

These semantics caution against deriving resource values from words alone. A phrase such as “large analysis” does not specify a safe memory ceiling. An explicit minimum-memory statement can be represented as a constraint, but a vague workload description is not equivalent to an observed peak. Similarly, the presence of “GPU” in a request is not evidence that a suitable accelerator is available. P2 can filter catalog candidates according to declared capabilities and configured policy; it cannot reserve a live device or override cluster admission. If no catalog entry satisfies an explicit requirement, the correct result is a no-feasible-candidate condition or user-directed manual path, not a fabricated configuration.

Kubernetes will make later admission and placement decisions from current cluster state. A candidate that passes P2 constraints can still fail because of quota, taints, affinity, node pressure, admission policy, registry access, or a race between preview and spawn. The design therefore needs a clear handoff: P2 emits an allowlisted configuration proposal; the normal platform lifecycle remains authoritative.

2.1.2. Resource estimation and workload-aware management

Resource-management systems often rely on runtime observations, workload history, trial executions, or service-level objectives. Vertical Pod Autoscaling can recommend or apply requests based on observed usage. Autopilot and Resource Central study workload-aware management at larger operational scales. Ernest and CherryPick use sampled runs or optimization to predict performance and identify configurations [24, 34, 6, 36, 1].

These systems demonstrate that resource selection is a substantive systems problem, but their evidence differs from a user’s pre-run task description.

Intent-Spawner is complementary in scope. It operates at the moment when the user chooses an environment and may have no profile-specific execution history. It can recommend a curated option based on expressed intent and known catalog metadata, then evaluate whether that option was adequate using independent workload experiments. It must not claim to predict exact demand from intent. The term *safe envelope* in E4 refers to an experimentally declared range under a particular workload, harness, and environment; it is not a universal upper bound for all notebooks with a similar description.

A reliable rightsizing experiment needs to distinguish requests from limits, establish a repeatable workload family, observe resource use under a stated environment, include failures such as OOM kills, and preserve trial-level provenance. Repeated runs are useful to estimate variability and stability. They do not become additional semantic workload families. This distinction is central to the Protocol-v5 unit-of-analysis policy.

2.2. Intent and natural-language configuration

Intent-based systems describe a desired outcome at a higher level than the mechanism that realizes it. The IETF intent-based networking framework distinguishes intent translation from fulfillment and assurance [4]. Surveys of intent-based networking identify similar concerns across domains: translating an abstract request, validating it against policy, activating a solution, and evaluating the result [25]. Natural-language interfaces to system configuration face a related issue [26]: free-form expression is convenient, but translating it directly into executable commands can yield invalid or unsafe actions.

The Intent-Spawner design narrows the action space. A user describes a workload; the system maps the description to typed fields and selects from a finite catalog. It does not generate arbitrary Kubernetes manifests. The catalog and policy remain administrator-controlled. This makes validation tractable and supports human inspection before the decision is applied. Related work on intent-driven cloud resource design explores

translation from goals to resource descriptions [37], while P2 focuses on ranking Jupyter environments from curated candidates and preserving a pre-spawn confirmation boundary.

Structured extraction is important even when the original input is natural language. Fields such as required capabilities, excluded capabilities, minimum memory, accelerator requirement, and task tags can be validated and consumed independently. The structured representation creates an explicit place to record uncertainty and source. A guessed capability should not silently become equivalent to a user-stated hard requirement. The actual implementation validates a versioned schema and uses deterministic downstream rules; this constrains, but does not eliminate, upstream interpretation errors.

2.2.1. Sparse, dense, and fused retrieval

Sparse retrieval methods such as BM25 score lexical overlap with term-frequency and document-length normalization [19]. This is useful for exact technical tokens, library names, and framework identifiers. Its limitation is vocabulary mismatch: a user may describe “table cleanup” while a candidate is indexed under a library name or “dataframe transformation.” Dense retrieval maps text to vectors and can retrieve semantically similar descriptions despite surface variation. Sentence-BERT and Dense Passage Retrieval are prominent examples of learned embedding-based retrieval [32, 16]. Dense matching can, however, blur distinctions among technical terms or return semantically related candidates that do not satisfy a mandatory capability.

Hybrid retrieval uses complementary signals. P2 combines BM25 and dense-retrieval rankings, then applies Reciprocal Rank Fusion (RRF). RRF combines ranks rather than requiring the scores from two retrieval systems to share a calibrated scale [5]. In a typical formulation, each candidate receives a fused score equal to the sum of reciprocal rank contributions, with a positive smoothing constant k :

$$\text{RRF}(c) = \sum_{r \in \mathcal{R}} \frac{1}{k + \text{rank}_r(c)}.$$

Here $\mathcal{R}$ is the set of retrieval lists. This is an established rank aggregation technique; the contribution is its use in a controlled recommendation pipeline, not the formula itself.

Retrieval is candidate generation, not final feasibility. The top retrieved candidate may still be invalid under an explicit requirement or policy. P2 therefore applies deterministic constraint evaluation after retrieval and only ranks the remaining feasible set. Retrieval recall and final recommendation quality are separate measurements: low recall limits what downstream ranking can recover, while high recall alone does not imply that the top recommendation is appropriate.

2.2.2. Constraint-aware recommendation and decision support

Constraint-based recommenders separate hard requirements from soft preferences [9]. For Intent-Spawner, hard constraints include requirements that make a candidate unusable, such as an explicitly required framework or accelerator that the candidate does not declare. Soft preferences can express task similarity, preferred resource class, or other non-mandatory compatibility signals. The distinction matters because a weighted score should not allow a strong semantic match to compensate for violating a mandatory safety or capability condition.

After infeasible candidates are removed, P2 produces a deterministic order. A reproducible order requires fixed component versions, stable candidate identifiers, configured ranking weights, and deterministic tie-breaks. Ranking metrics such as Mean Reciprocal Rank and normalized discounted cumulative gain characterize the location of acceptable alternatives in a list [11, 14]. They do not capture whether the user understands a recommendation, whether the environment actually executes the task, or whether the experience is faster.

Decision-support literature emphasizes the allocation of authority between a system and its user [29, 13]. Intent-Spawner follows a human-in-the-loop model: it explains the candidate and its attributes, invites confirmation, preserves editing and allowlisted override, and does not spawn solely because a model produced a top result. This control

boundary also supports error recovery: a user can reject an unhelpful suggestion before the environment is created.

2.3. Human-centered evaluation

Recommendation systems are not evaluated adequately by ranking metrics alone. User-centered frameworks distinguish accuracy, perceived usefulness, confidence, transparency, satisfaction, and effort [31, 18]. For this system, E3 is designed as a within-participant comparison between B0 manual selection and P2-assisted selection. Candidate outcomes include task completion time, correctness or acceptable choice, confidence, perceived workload, and usability measures such as the System Usability Scale [3, 35].

A crossover design can reduce between-participant variance by exposing each participant to both conditions, but it needs counterbalancing, clear task equivalence, training, and analysis that accounts for participant and task effects. Repeated task variants cannot be treated as independent people. Moreover, consent and privacy safeguards are required because task logs may contain identifying details. No such participants have been measured in the current E3 package. Therefore, this chapter describes the planned construct and instrument, not a usability result.

2.3.1. Container images, layers, and storage accounting

OCI images are assembled from content-addressed layers described by a manifest. A layered design allows images to share immutable content, but the meaning of “size” depends on what is counted. Summing every layer descriptor occurrence across image manifests gives a logical sum; summing unique digests gives unique layer bytes for that catalog prefix. The two quantities differ when layers are shared or repeated. Docker storage documentation and studies of container distribution discuss this reuse and its operational implications [28, 7, 10, 2].

Unique manifest-layer bytes are not the same as measured node disk consumption. Runtime snapshotters, unpacked filesystems, compression, garbage collection, local

caches, and registry behavior affect actual storage. E5 therefore labels its size domain as compressed OCI manifest layer bytes and reports the target platform. The interpretation remains limited to the measured catalog and does not generalize automatically to a production cluster or to a larger catalog that was unavailable for measurement.

2.3.2. Evaluation concepts and statistical discipline

A recommendation benchmark needs a clear definition of a query, a candidate, a relevant set, and the independent unit. If a workload family is written in English, Vietnamese, abbreviated text, and several paraphrases, those variants remain expressions of the same semantic task. Treating each variant as independent would make confidence intervals artificially narrow. The Protocol-v5 design therefore aggregates at family level and uses repeated executions only to characterize execution variability. This is consistent with the general principle that the analysis unit must match the source of independent variation.

For a ranked list, reciprocal rank rewards the position of the first acceptable item [11]. Discounted cumulative gain assigns graded relevance and discounts lower positions; normalized DCG divides by an ideal ranking to permit comparison across queries with different relevant-set sizes [14]. Hit@K answers whether at least one acceptable option appears in the first K positions. These metrics answer different ranking questions, and none directly measures user value. The primary metric must be chosen before confirmatory execution. Secondary metrics should be reported with their definitions and uncertainty, not selected after seeing which one favors a method.

Paired analysis is suitable when P1 and P2 receive the same workload families. Pairing removes some between-family difficulty variation, but does not remove the need for enough independent families. If several hypotheses are tested, multiplicity control such as Holm’s sequential procedure limits family-wise error [12]. Bootstrap intervals can express uncertainty, but the resampling scheme must preserve the family as the sampling unit [8]. Small samples and tied outcomes may make conclusions inconclusive; an honest inconclusive result is preferable to a post hoc metric change.

Human studies have a different unit. A participant completing multiple tasks contributes repeated measures, and task order may create learning or fatigue effects. A crossover study should counterbalance condition order, report exclusions, and model participant-level dependence. A timing difference can be practically small even when statistically detectable, while a usability score can summarize perceived interaction without proving task correctness. The constructs should be reported separately. E3 is consequently planned as an independent participant-based stream rather than an extension of offline ranking evaluation.

Resource trials have yet another unit structure. A workload family can be repeated under resource limits to identify failure boundaries, but the trials are nested within family and environment. OOM events are meaningful outcomes, not discarded outliers. Safe-envelope derivation requires an explicit rule (for example, a minimum configuration meeting a declared success and headroom criterion), and should distinguish observed limits from recommended profile policy. CPU throttling, memory peaks, process exit, and measurement coverage should be retained at trial level. The current local run count must not be mistaken for hundreds of independent task types.

2.4. Catalog quality and metadata

A retrieval system can only rank the information represented in its candidate corpus. Catalog metadata therefore affects both relevance and feasibility. Candidate descriptions should identify supported frameworks and libraries, base image, resource profile, accelerator class, and applicable limits with controlled names where possible. If an image can execute a capability that its metadata omits, the system may under-recommend it; if metadata claims a capability that the image lacks, it may recommend a broken environment. The E5 functional probe stream addresses part of this metadata-to-execution relationship, but the current bounded probes do not certify all image functionality.

Metadata maintenance is an operational responsibility. Image updates can alter package versions, image digests, size, and capabilities. A catalog release should bind its

metadata to immutable references and be tested before activation. Search-index generation should include its source catalog version and checksum. Otherwise, two runs described as the same P2 configuration may silently differ in candidate content even if code and weights remain fixed. This is why provenance includes catalog and index versions alongside source revision and dataset identity.

Candidate coverage also places an upper bound on recommendation quality. If no approved candidate has a required framework, no ranking algorithm can make the requirement feasible. Adding candidates changes the recommendation space and image-storage footprint and therefore changes the experimental object. Catalog scale experiments should use actual approved images, retain their provenance, and avoid implying results for scales that were configured but not executed. In the current E5 storage study, the catalog contains four approved images; the larger configured scales remain unexecuted.

2.4.1. Operational reliability and user trust

The recommendation endpoint participates in a pre-spawn workflow, so latency, availability, explainability, and recoverability all matter. A user may tolerate a short delay in exchange for a useful comparison, but should not be left with a spinner or ambiguous failure if the retrieval service is unavailable. A sensible interaction can surface a concise explanation of matched task terms, explicit requirements, resource attributes, and any constraint-based exclusion. Such an explanation does not need to expose model internals; it should show the grounds that matter to the user’s decision.

Trust is earned by preserving a meaningful override and by making uncertainty visible. If a capability is not verified in the catalog, the interface should not imply that it has been tested. If no candidate satisfies a GPU requirement, the user should see that infeasibility. If a local resource envelope is used to inform a profile, the interface should not label it as a guarantee for workloads it has not measured. These properties connect technical correctness with human control, and they are why preview and confirmation are part of the method rather than decorative interface elements.

The deployment lifecycle also raises reliability requirements. Preview state must survive the route from the UI request to the process handling the spawn hook. A short TTL limits stale decisions, and single-use consumption prevents replay. Multi-replica deployment requires a shared state store or another mechanism that offers equivalent consistency. Failure to resolve the stored candidate should fail closed and provide a recoverable error. These are standard distributed-application concerns, but they are especially visible here because the system must apply exactly the option the user reviewed.

2.4.2. Design trade-offs in a curated environment catalog

A finite catalog simplifies enforcement but creates a coverage trade-off. A very small catalog is easier to secure, test, and maintain; it may not cover specialized libraries or resource combinations. A very large catalog may include more matches but makes image maintenance, user choice, storage accounting, and compatibility review harder. The recommendation system does not eliminate this trade-off. It can help users navigate a catalog, but only administrator decisions can add supported capabilities or approve new images. Catalog expansion should therefore be evaluated jointly for user-facing coverage and operational cost.

There is also a trade-off between semantic flexibility and determinism. Natural language accepts many valid ways to express the same task, while infrastructure configuration requires precise values. Structured extraction can bridge these forms, but each inference introduces uncertainty. The system should interpret softly where only a preference is implied and enforce strictly where a requirement is explicit. The preview gives the user a chance to resolve ambiguous values before creation. This is a domain-appropriate compromise: more flexible than exact keyword matching, yet more bounded than open-ended manifest generation.

A second trade-off concerns resource conservatism. A recommendation that uses more memory than necessary can reduce failure risk but may reduce cluster density or violate quotas. A recommendation that is too small can cause failure, restart, and lost work. Without prior execution measurements, the system cannot calculate this trade-off

exactly. Resource profiles should thus be drawn from administrator-reviewed options and backed by measured envelopes where available. The confidence of an envelope depends on workload construction, repetitions, resource measurement, and target cluster conditions. A local synthetic workload can help calibrate a prototype but cannot stand in for every user notebook.

Finally, the system must balance explanation detail with interface burden. Listing every retrieval feature or model score would make the interface harder to read without improving the decision. A concise reason should instead expose task tags, validated requirements, resource settings, and compatibility evidence that matter to the user. If a property has not been verified, that status should be visible. This choice keeps explainability grounded in data rather than presenting a persuasive narrative that may overstate model certainty.

2.4.3. Terminology and construct boundaries

Several terms in this thesis have deliberately narrow meanings. A *candidate* is an administrator-defined profile-image option, not an arbitrary Kubernetes object. A *recommendation* is a ranked candidate list or its top selection before user confirmation. A *confirmed decision* is the option bound to the user’s preview state and applied by the pre-spawn hook. A *workload family* is a semantic task grouping that can have multiple prompt realizations. A *prompt variant* is one text form of that family. A *trial* is a repeated execution under a particular environment and setting.

The word *feasible* is local to declared constraints and catalog metadata. It does not mean the Pod is guaranteed to schedule. The word *safe* in a measured resource envelope is conditional on an explicit workload and environment, rather than a universal safety guarantee. *Image functionality* refers to passing the probes that were actually executed, not exhaustive proof that every package works. *Storage savings* in E5 means a difference between logical and unique compressed layer bytes, not observed disk savings on nodes. These definitions prevent terms that sound broad in ordinary language from being read as stronger technical claims than the measurements support.

This terminology also clarifies what different evidence can resolve. E1/E2 can assess candidate relevance and language robustness under a frozen dataset. E3 can assess user outcomes in a particular study population. E4 can estimate resource outcomes under measured workload families and infrastructure conditions. E5 can assess selected functional probes and manifest-layer accounting. A positive outcome in one stream cannot compensate for missing evidence in another because the constructs and units differ.

2.4.4. Research gap and synthesis

Adjacent work provides notebook service primitives, cluster resource control, telemetry-driven rightsizing, intent translation, retrieval, constraint-aware recommendation, user evaluation, and image distribution. The repository-informed gap is a narrow integration question: how can workload language be mapped to a small approved Jupyter environment catalog before spawn while retaining deterministic policy enforcement, user confirmation, and evidence boundaries?

P2 addresses that question as a bounded decision-support system. Its architecture joins typed intent, hybrid retrieval, hard feasibility, deterministic preference ranking, trusted candidate resolution, and a stored confirmation. Its evaluation deliberately avoids collapsing distinct claims into one accuracy score. The method is positioned as a practical prototype whose recommendation and operational consequences require further confirmatory evaluation. No current result establishes that it improves all relevant outcomes or that its behavior transfers to every JupyterHub catalog.

Chapter 3.

Proposed Method

3.1. System taxonomy and design objective

The thesis distinguishes four systems. **B0** is the default manual JupyterHub selection workflow and generates no ranking. **P1** is the frozen existing rule-based recommender. **P2** is the principal proposed method: structured workload intent, hybrid retrieval, deterministic constraints, and ranking. **P3** is a grounded large-language-model reranking extension, considered only if it passes a development gate. P3 did not satisfy the retained-branch criterion in the current development evidence and is not part of the main deployed path.

P2’s objective is to return an inspectable, policy-compliant ordering of candidates in a finite administrator-owned catalog. The output is advisory until the authenticated user confirms it. A candidate that fails a mandatory constraint is not made acceptable by high retrieval relevance. The design objective can therefore be summarized as a constrained ranking problem:

$$\text{rank}_{P2}(q, C) = \text{sort}_{\text{det}}\{c \in C : H(q, c) = 1\},$$

where q is validated structured intent, C is the active catalog, H is the conjunction of hard feasibility predicates, and sort_{det} orders feasible candidates by configured preference scores with stable tie-breaking.

3.1.1. Structured intent representation

The raw request is first parsed into a versioned *StructuredIntent*. This representation keeps free text available for retrieval while separating explicit constraints from inferred

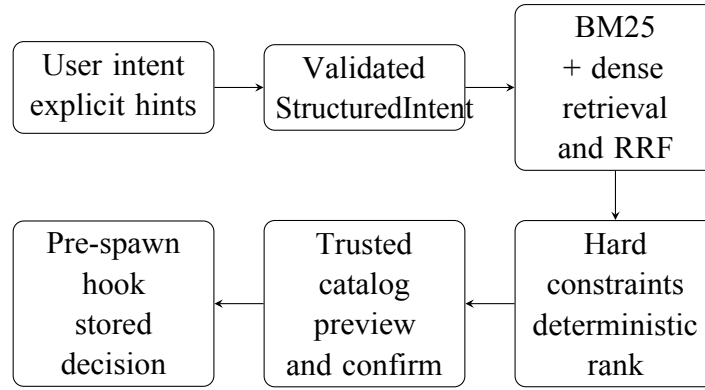


Figure 3.1. P2 transforms a bounded intent into a confirmed catalog choice before the ordinary Kubernetes spawn path.

task descriptors. Representative fields include canonical task terms, required capabilities, excluded capabilities, optional framework and library hints, minimum resource requests when explicitly stated, accelerator requirements, dataset/workload hints, and provenance indicating whether a value was directly supplied or inferred. A schema validator rejects malformed types and bounds before retrieval begins.

The distinction between explicit and inferred fields prevents a soft semantic interpretation from becoming an accidental mandatory requirement. For example, “I want to explore a medium-sized CSV” may contribute data-analysis terms and an approximate workload tag, but it does not establish a measured memory requirement. In contrast, “the notebook must have at least 8 GiB” can be represented as a numeric minimum and checked against candidate metadata. If the language parser is uncertain, the system should preserve that uncertainty or ask the user to edit the preview rather than silently turning it into a hard constraint.

StructuredIntent is an interface contract between language interpretation and candidate ranking. Its schema version, normalization behavior, and field semantics need to be frozen for a confirmatory run. If a schema changes after development, the dataset and implementation provenance must make that change explicit. The experiment protocol treats a change in parsing behavior as a material change to the system, not as an invisible data-cleaning adjustment.

3.1.2. Hybrid candidate retrieval

Each candidate in the administrator catalog has searchable text derived from its stable identifier, description, capabilities, framework/library metadata, resource class, and other approved attributes. P2 evaluates a lexical retrieval channel and a dense semantic retrieval channel. BM25 helps preserve exact technical vocabulary; dense retrieval helps with paraphrase and semantically related expressions [33, 32, 16, 27]. Each channel emits an ordered candidate list. RRF fuses the ranks:

$$S_{\text{hybrid}}(c \mid q) = \sum_{m \in \{\text{BM25, dense}\}} \frac{1}{k + r_m(c)},$$

where $r_m(c)$ is the candidate's one-based rank from method m , and a missing candidate contributes zero. The fusion constant, retrieval depth, embedding model, text construction, and index version are fixed configuration inputs that must be recorded.

Hybrid retrieval is intentionally a shortlist generator. It does not decide whether a candidate is policy-compliant and does not treat the dense score as a probability. This separation allows the system to combine complementary retrieval behavior without allowing similarity to overrule explicit requirements. E1/E2 separately record retrieval recall and final recommendation metrics so that a retrieval failure is not misattributed to downstream ranking.

3.2. Hard constraints and deterministic ranking

Let $R(q)$ be the candidates returned by retrieval. P2 evaluates each candidate against hard constraints $h_j(q, c) \in \{0, 1\}$. Feasibility is

$$H(q, c) = \prod_{j=1}^J h_j(q, c).$$

A candidate is feasible only if every required predicate is true. Examples include required capability present, prohibited capability absent, explicit minimum resource met, accelerator class compatible, and candidate enabled by current administrator policy.

Missing metadata for a required property is not equivalent to a confirmed match; the implementation must use a conservative failure or an explicitly documented unknown policy.

For candidates that remain feasible, a deterministic ranking function combines configured signals such as hybrid retrieval rank, capability fit, workload category, and resource preference. The precise weights and normalization are implementation configuration and are not tuned using confirmatory cases. Stable ordering by candidate identifier resolves exact ties. The method does not present its score as a calibrated probability of task success.

The empty feasible set is a meaningful system outcome. It should be surfaced to the user with the reason that no current catalog candidate satisfies the request. The user can revise the request or choose an existing allowlisted manual profile according to deployment policy. P2 must not weaken a hard constraint to force a non-empty result. This behavior is especially important for unsupported accelerator, memory, or software requirements.

3.2.1. Trusted candidate corpus and policy boundary

The user's language is untrusted input. Candidate identifiers, image references, profile mappings, constraints, and policy are administrator-owned data. The recommender returns an identifier, not an arbitrary container reference or raw Kubernetes object. The integration resolves that identifier from the current trusted catalog, verifies that the candidate is enabled and allowlisted, and checks the catalog and policy version before preparing a preview. Images are expected to be digest pinned for immutable identity.

This boundary constrains the effect of parsing and ranking errors. A mistaken interpretation can recommend the wrong approved profile, but it should not authorize an unapproved image or inject arbitrary Pod configuration. Kubernetes admission remains the final enforcement point. A preview token is bound to the authenticated user and the stored decision; it is single use and has a time-to-live. Changes to the request invalidate the old decision and require another preview. The pre-spawn hook applies the stored

confirmed selection rather than rerunning the model at spawn time, so the decision visible to the user is the decision applied by the integration.

3.2.2. Preview, confirmation, and pre-spawn binding

The interaction is organized as a state transition:

1. The authenticated user submits an intent and optional explicit hints.
2. P2 validates the input and computes an ordered candidate set from a versioned catalog.
3. The service stores the preview state, including the selected candidate identifier, rationale, constraints, catalog/policy versions, user identity binding, creation time, and expiry.
4. The interface displays the recommendation and relevant properties. The user may confirm, edit the request, or choose an allowlisted alternative.
5. Confirmation consumes a single-use token associated with that stored state.
6. The JupyterHub pre-spawn hook verifies user, expiry, token status, policy, and catalog identity, then applies the stored configuration.

The stored-state design prevents time-of-check/time-of-use drift from silently changing the choice after confirmation. It does not make cluster admission atomic with preview. If the policy or catalog version changes, the stored choice can be rejected and the user asked to generate a fresh preview. If the Pod fails to schedule, ordinary Kubernetes diagnostics apply; that failure is not evidence that the semantic recommender made a wrong ranking decision.

3.3. Rationale for the selected architecture

The architecture chooses hybrid retrieval because the catalog has both exact technical vocabulary and human-readable descriptions. A sparse method is responsive to framework

and package names; a dense method can help when task wording differs from candidate descriptions. Rank fusion avoids treating raw relevance scores from different systems as directly comparable. Deterministic feasibility comes after retrieval because a hard requirement should not be represented as a compensable preference. Finally, catalog resolution occurs after the ranking stage because the ranking result must never itself become authority to choose an arbitrary image.

Each choice has a limitation. BM25 depends on useful tokenization and metadata. Dense retrieval depends on the embedding model and may blur near-neighbor concepts. Fusion depends on the retrieval depth and constant. A fixed constraint set may not capture every operational rule. Deterministic ranking can be repeatable while still being wrong if the catalog is inaccurate. The architecture does not remove these limitations; it creates separate interfaces and versions so that they can be tested and updated without obscuring what changed.

The preview and confirmation step is also a deliberate design choice. An automatic spawn could reduce interaction, but it would transfer authority to a system whose interpretation may be incomplete. A preview makes the recommendation useful without requiring users to trust it blindly. The state token also ensures that confirmation applies to the option displayed. This additional interaction has a cost and must be tested with participants. Since E3 has not been executed, the thesis cannot yet claim that the design strikes the optimal balance between speed and user control.

3.3.1. Optional dynamic sizing and reprovisioning

The repository contains prototype extensions for resource profile generation and intent-aware reprovisioning. Dynamic sizing applies configured bounds and policy to explicit workload information and profile metadata. It is not the principal P2 ranking method and is not an exact demand predictor. Any generated size must respect administrator minima/maxima and be distinguished from values derived from independent telemetry. The extension should be evaluated separately because it changes the action space and the resource-safety claim.

The reprovisioning design supports a user requesting a different environment after a session exists. It is a stop-and-recreate workflow with explicit lifecycle and persistent-volume considerations, not in-place migration of a running process. User files must be preserved according to storage policy; kernel state and in-memory objects cannot be assumed to survive recreation. The current thesis treats this feature as a prototype extension and makes no claim of transparent migration or production-grade availability.

3.3.2. Ranking pipeline and score interpretation

The method can be represented as a sequence of typed transformations. Let x denote the original request, N_v a versioned intent normalizer, $I = N_v(x)$ the structured intent, T_m the catalog text for candidate m , and E the active catalog/index version. Retrieval computes two ordered lists, $L_s = \text{BM25}(I, E)$ and $L_d = \text{Dense}(I, E)$. The fused list is $L_f = \text{RRF}(L_s, L_d)$. Constraint evaluation produces the feasible set $F = \{c \in L_f : H(I, c) = 1\}$. A configured preference function $P(I, c)$ orders F , with deterministic tie-breaking on stable candidate identifiers. This sequence makes it possible to attribute an error to interpretation, candidate recall, feasibility metadata, or final ranking rather than treating the recommender as one opaque score.

A high fused rank means that the candidate appeared near the top of one or both retrieval lists; it does not mean the candidate has a stated probability of success. Similarly, the preference score orders candidates under the configured policy and is not calibrated confidence. The interface should avoid percentages that imply calibration unless a separate calibration analysis has been performed. A compact rationale can report exact-match terms, semantic task tags, and constraints satisfied, while separately showing any uncertain inferred field.

This architecture encourages a conservative rule for missing metadata. If a user explicitly requires an accelerator and the candidate record lacks accelerator support, the system should treat it as not demonstrated and exclude it, unless the policy explicitly defines an unknown state that triggers user review. It should not impute capability from a nearby name or from the embedding similarity of the description. This conservative

handling may lower top-one coverage when metadata is incomplete, but it keeps a hard requirement from being masked by a soft signal.

3.4. Algorithmic outline

Algorithmically, P2 has two responsibilities: rank feasible items and produce a result that can be applied later without recomputation. The first responsibility is a deterministic function of the normalized request, catalog, index, and frozen configuration. The second responsibility is a state-management function that binds the selected candidate to user confirmation and the hook.

The recommendation routine performs schema validation before accessing the index. It then queries each retriever with a fixed depth, merges candidate identifiers, computes fused ranks, and asks the constraint evaluator to return a structured feasibility record for every candidate. The output records candidates removed by each hard constraint, when that explanation can be disclosed safely. The remaining list is sorted by a fixed preference tuple, such as descending configured score followed by ascending stable identifier. All inputs that can affect this order are versioned.

The preview service stores the exact candidate identifier and decision context. The confirmation endpoint atomically consumes the token and marks the preview confirmed. On spawn, the hook verifies the authenticated user binding, expiration, token state, policy version, and catalog resolution. If any check fails, the hook does not substitute a newly ranked option because that would violate the preview contract. It fails with a clear requirement to regenerate or manually select an approved profile. This separation supports audit logs that connect an observed Pod configuration to the decision the user saw.

3.4.1. Threat model

The threat model includes accidental malformed input, adversarial text attempting to name an unapproved image, replay of an old confirmation token, cross-user token use, stale catalog references, metadata errors, retrieval outages, and concurrent consumption.

The core security response is to treat user intent only as data, resolve all selections through administrator-owned identifiers, validate policy again at the hook, bind tokens to users, and consume tokens atomically. The threat model does not assume that the recommender can defend against a compromised Hub, cluster administrator, catalog signing key, or Kubernetes control plane.

Prompt injection is less central in the P2 path because P2 does not ask an LLM to generate deployable configuration. If an optional P3 component is introduced, its output must be limited to reranking identifiers from the already feasible candidate set. It must not add a new image, weaken a constraint, or alter resource values. The P3 gate exists partly because extra model complexity creates failure modes and operational cost; a branch that lacks measurable ranking headroom should not be retained.

Metadata poisoning is addressed through code review, versioned catalog provenance, digest-pinned image references, and bounded functional tests. These do not prove the image is harmless. They establish which artifact was tested and constrain mutable tags. Runtime sandboxing and container security remain deployment responsibilities.

3.4.2. Privacy, auditability, and reproducibility

The system should retain only the information necessary to explain a decision and support audit. Logs should avoid raw participant identifiers and unnecessary full free-text prompts. User-study records must be pseudonymized, and participant data must be governed by consent and retention rules. Operational provenance should record timestamp, source revision, protocol version, dataset checksum, backend and index versions, and relevant environment identity. Recommendation output, raw observations, derived metrics, statistical conclusions, and interpretation are separate artifacts.

Reproducibility also requires frozen comparator versions and a protected confirmatory split. No prompts, retrieval parameters, candidate metadata, thresholds, ranking weights, constraints, or P3 settings may be tuned on final confirmatory cases. Workload family is the semantic independent unit. Repeated stochastic executions estimate variability; prompt paraphrases and repeats do not inflate the independent sample

count. These requirements make the evaluation design part of the method rather than an afterthought.

Chapter 4.

Implementation and Experiments

4.1. Implemented workflow and evidence design

4.1.1. Repository implementation

The repository implements the recommendation path as separable components. Structured-intent parsing and validation provide typed inputs; hybrid retrieval produces candidate lists; a constraint evaluator marks infeasible candidates; P2 applies the ranking policy; a candidate corpus resolves trusted metadata; policy checks enforce allowlists; and a JupyterHub integration provides preview and pre-spawn behavior. Protocol-v5 code provides schemas, runners, validation, statistical analysis, and report generation. This division supports focused inspection and helps preserve the historical Protocol-v4 infrastructure rather than replacing its evidence.

The request path is intentionally bounded. It begins with authenticated user input, not a batch of arbitrary cluster objects. Candidate text is indexed from administrator-owned metadata. The retrieval backend can be replaced only with a versioned configuration and matching provenance. The output is a stable candidate identifier and explanation data; the hook obtains final profile and image values from trusted candidate state. This reduces the chance that a free-text request can become an unreviewed image reference or raw resource dictionary.

The preview and spawn path preserves the human decision. A user can see the recommendation, check its resource and capability properties, confirm or edit, and use only an approved override. A user-bound, short-lived, single-use token links the confirmed choice to the hook. Edits invalidate the earlier token. The hook does not recompute a

possibly different recommendation. This makes application behavior inspectable and permits stale catalog or policy state to be rejected explicitly.

4.1.2. Failure handling and safety properties

A robust path must handle malformed intent, retrieval service errors, empty results, catalog drift, expired tokens, replayed tokens, unauthorized users, and changed policy. Input-validation errors should be distinguishable from “no feasible candidate.” Retrieval failures should not silently be represented as confident low-quality recommendations. Expired or consumed tokens should fail closed. A missing or changed candidate should require a new preview instead of applying stale state. These cases are part of correctness because the recommender operates in a user-facing control path.

The method preserves several invariants: only administrator-approved candidates are resolvable; image identity is immutable when digest-pinned references are used; hard constraints cannot be bypassed by ranking score; the preview is bound to the user and the candidate state; the hook applies a confirmed decision; and Kubernetes admission remains authoritative. These invariants are testable at component and integration boundaries. The thesis does not infer production security from unit-level success; deployment still requires review of authentication, shared state, multi-replica behavior, secrets, network exposure, and admission controls.

One deployment concern is persistence of preview state. In-memory storage can be adequate for a single-process prototype but is not sufficient for a multi-replica Hub unless state is shared or requests are reliably routed. Productionization should use a durable shared store with expiry and atomic single-use consumption. This is an implementation limitation to resolve before multi-replica deployment.

4.1.3. Protocol-v5 scope and empirical status

Protocol-v5 separates development, final confirmatory evidence, raw observation, derived metrics, and interpretation. For this thesis, the repository’s streamlined experiment guide permits technical reporting of the available E1, E4, and E5 measurements

without the administrative freeze and custodian-signature workflow used for a formal confirmatory package. Accordingly, this chapter reports the observed quantities and their actual local scope. The distinction matters: an observation is a measurement that occurred; it does not automatically establish a general effect or a confirmatory hypothesis decision. The consolidated thesis-results registry remains marked INCOMPLETE / NOT_EXECUTED for its stricter registered claims, and that status is reported separately rather than used to erase measured results.

Protocol-v4 evidence remains historical/formative and is not overwritten or relabeled as Protocol-v5. Versioned results record provenance and checksums. Workload family, not paraphrase, is the semantic unit for offline recommendation. A participant is the independent unit for human outcomes. Repeated stochastic executions characterize stability and runtime variability, not extra accuracy samples. B0 has no ranking output; therefore, MRR, nDCG, and Hit@K do not apply to B0. These distinctions prevent unlike evidence and units from being pooled.

Table 4.1. Protocol-v5 experiment streams and evidence scope

Stream	Question	Construct	Evidence reported here
E1	RQ1	Family-level recommendation quality, P1 vs P2	Development split: 18 families, 83 variants; observed estimates and tests
E2	RQ2	Robustness across prompt forms	Development split: macro SRR and worst-family rate
E3	RQ3	Human time, choice, confidence, workload, usability	No participant observations; protocol only
E4	RQ4	Local resource envelope and Pod outcomes	OrbStack: 421 trials, 20 families
E5	RQ5	Image capability probes and OCI layer bytes	Docker probes and four-image linux/arm64 accounting
P3	RQ6	Development gate for LLM reranking	11-family gate; branch not retained

The report chain separates raw observations, derived statistics, and interpretation. A Pod record is not itself a safe envelope; a confidence interval is a derived quantity; and neither should be conflated with a claim about another cluster. Provenance includes timestamp, git revision and dirty state, protocol version, dataset checksum, backend versions, catalog/index versions, and environment identity where relevant. E1/E2 was generated at 2026-09-28 03:06 UTC on the v5-development split; the E5 functional and storage packages were generated a few minutes later and identify their own sources and catalog. The E4 package names the local OrbStack Kubernetes context. Different packages can use different metrics or processing paths even when they share recommendation inputs.

4.1.4. Reproducibility and interpretation rules

The repository includes schema validation, experiment runners, provenance records, result manifests, validators, analysis outputs, plots, and reports. A reader can trace a reported number through this chain and inspect the assumptions used to calculate it. A dirty working tree is not by itself proof that a result is invalid, but it requires identifying the source snapshot and avoiding confusion with a later clean revision. Reports should retain their original source labels and measurement conditions.

A final thesis build depends on the source tree, TeX engine, class version, fonts, image assets, and bibliography. Generated intermediate files are not source evidence. Figures copied from experiment packages retain captions that state what was measured. When results disagree across packages, the correct treatment is to preserve the separate definitions and denominators, then investigate the discrepancy; averaging the values would create a quantity no report measured.

4.2. Offline recommendation and robustness results

4.2.1. E1: family-level recommendation quality

The E1 report evaluates 18 independent workload families and 83 prompt variants on the v5-development split. Estimates are family macro-averages, with 95% percentile

bootstrap confidence intervals; the semantic unit is the family, not the individual prompt. The report provides paired p-values for the listed comparisons. Table 4.2 transcribes the primary reported values.

Table 4.2. Observed E1 development estimates for P1 and P2

Metric	P1 estimate [95% CI]	P2 estimate [95% CI]	Reported p
JointAccept@1	60.0% [34.5, 81.8]	65.0% [43.6, 85.0]	0.7617
Profile acceptability	83.6% [63.6, 100.0]	74.1% [54.5, 90.5]	0.4062
Image acceptability	63.6% [38.2, 87.3]	87.3% [74.5, 96.4]	0.2344
Constraint violations	38.2% [16.4, 63.6]	21.8% [5.5, 43.6]	0.4062
Unsupported detection	0.0% [0.0, 0.0]	50.0% [29.2, 75.0]	N/A
Latency execution	per 0.000 s [0.000, 0.000]	0.002 s [0.002, 0.002]	<0.001

The point estimates describe different trade-offs rather than a uniform ordering. P2’s observed JointAccept@1 estimate is five percentage points higher, its image acceptability estimate is higher, and its constraint-violation estimate is lower. Its profile acceptability estimate is lower. The reported p-values for JointAccept@1, profile acceptability, image acceptability, and constraint violations are all above 0.05. Thus the table records descriptive differences in this development sample; these tests do not establish a statistically significant difference on those outcomes. Unsupported detection is reported without an inferential p-value. The latency figure is a rounded summary at millisecond-scale for P2 and sub-millisecond display precision for P1; the displayed zeros should be read as report rounding, not proof of zero computational cost.

The confidence intervals are broad, which is consistent with an evaluation containing 18 independent families. The family-level design prevents the 83 prompt variants from being treated as 83 unrelated workloads. Results are sensitive to the family composition and to the operational definitions of acceptability and constraint violations. In particular, the report’s family macro-average weights each family equally; it is not a request-volume-weighted estimate for a production Hub. The observed estimates are useful

for understanding the behavior of the current corpus and implementation, but should not be projected to an unknown catalog or population of user tasks.

4.2.2. E2: language and prompt robustness

E2 measures semantic robustness across reviewed-equivalent prompt forms. Its reported overall Semantic Robustness Rate (SRR) is an equal-weight macro mean over workload families, while worst-case family robustness asks what fraction of families succeeded on every tested equivalent variant. Table 4.3 presents the report values, including the per-form cells as published. N/A cells and zero-valued displayed cells are retained because the report does not provide valid denominators for reconstructing all of them.

Table 4.3. Observed E2 robustness metrics on the development split

Measure	P1	P2	Interpretation	
Overall SRR (macro)	56.8% [31.8, 79.5]	60.6% [36.3, 83.3]	Family mean	macro
Worst-case family robustness	0.0%	45.5%	All tested equivalent variants succeed	
Vietnamese cell shown	0.0% (N/A)	20.0% (N/A)	Denominator unavailable in report	
Canonical cell shown	0.0% (N/A)	0.0% (N/A)	Denominator unavailable in report	

P2’s overall SRR is 3.8 percentage points above P1 in this report, and the reported worst-case measure is also higher. These are bounded development observations, not estimates from a sealed final split. The per-form table contains N/A denominators for several categories, so it cannot support a reliable language-by-language ranking or reconstruction of a pooled success count. Robustness here also has a specific meaning: the benchmark’s reviewed-equivalent variants should preserve intent. It does not measure arbitrary user language, dialect diversity, accessibility, or interaction repair.

4.2.3. E5 recommendation metrics and cross-report reconciliation

The E5 functional report includes a separate gold-label correctness dimension over 83 cases: P1 matched the preferred label in 34/83 cases (41.0%), and P2 in 46/83 (55.4%). It reports these as both preferred and acceptable matches. These figures are not substituted for E1's profile, image, or joint acceptability measures. E1 and E5 differ in metric definitions and report lineage, so the quantities answer different questions and are kept in separate tables. In particular, the E5 label-matching rates should not be read as a corrected version of E1 or pooled with it.

Taken together, E1 and E2 show measurable behavior on the development corpus and expose where further reconciliation would be valuable. E1's largest positive P2 point-estimate difference is image acceptability, while profile acceptability moves in the other direction. E2 suggests better robustness for P2 on two aggregate summaries, but incomplete per-form denominators limit finer conclusions. The development evidence therefore motivates follow-up work on profile selection, prompt variation, and metric traceability rather than one headline declaration that a method wins.

4.3. Cluster and image observations

4.3.1. E4: observed local resource trials

The E4 package records 421 Pod trials across 20 workload families in the OrbStack Kubernetes context `orbstack:z2jh-context-demo` (as recorded in the experiment evaluation package). Of these, 413 completed successfully (98.1%) and 8 were OOM-killed (1.9%). These counts are direct package-level observations. The experiment is useful as a local comparison of tested workloads and resource configurations; the context is a local cluster and does not represent a production node fleet.

The report includes examples that show why resource selection has a two-sided objective. A 256 MiB static profile was insufficient for selected memory-intensive families such as `large_hash_map_allocation` and `categorical_encoding`. Conversely, a

2048 MiB profile is much larger than the 64 MiB reported safe envelope for examples such as `streaming_statistics` and `sparse_graph_traversal`. The supplied workload table records 1000m CPU and 64 MiB safe RAM for many families, while some families have no reported safe envelope and `large_hash_map_allocation` is associated with OOM thresholds at 384, 1024, 1536, and 1792 MiB. These are family-specific results under the run's conditions.

The same report narrative states that P2 guarantees 100% OOM avoidance, but the aggregate trial table records eight OOM events. This thesis therefore reports the count and family-level examples and does not repeat the universal guarantee. A family-specific selected envelope and an aggregate set of Pod trials may summarize different subsets or trial types; resolving their exact relationship requires linking each row to raw run records. The measured 98.1% aggregate completion is the appropriate package-level statement until that reconciliation is complete. The report's example of up to 96.9% RAM excess on selected light workloads illustrates the scale of one comparison, not an average saving across all trials.

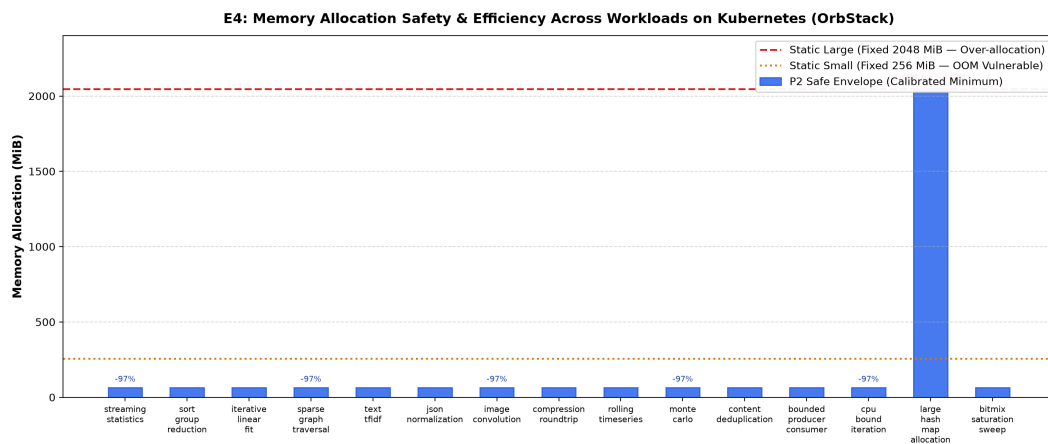


Figure 4.1. Locally observed memory envelopes and static profiles in the E4 package

For operational use, an envelope is meaningful only alongside its measurement definition: requested and limit values, runtime peak or sampled usage, workload completion criterion, repetition, and safety margin. A low observed peak does not imply that a larger input or concurrent process fits the same allocation. Kubernetes scheduling and enforcement further depend on node allocatable resources, requests, limits, QoS, and

competing pods. E4 supports examining these interactions in the observed context; it does not establish a universal optimizer or safe limit for other cluster configurations.

4.3.2. E5: functional capability probes

The E5 functional run reports 17 probe specifications and 166 evaluated recommendations, using Docker and an administrator catalog at version 2026-08-06.1. Its dimensions deliberately separate label correctness (A), catalog-declared capability coverage (B), and actual functional execution (C). Table 4.5 reproduces the key counts.

Table 4.4. E5 observed label, catalog, and functional dimensions

Dimension	P1	P2	Meaning
A: preferred-label match	34/83 (41.0%)	46/83 (55.4%)	Gold-label correctness
B: catalog capability coverage	58/83 (69.9%)	70/83 (84.3%)	Catalog declares requirements
C: functional eligible	65	76	Cases with executable probes
C: executed probes passed	65/65 (100%)	76/76 (100%)	Pass rate among executed eligible cases
Operational adequacy	58/83 (69.9%)	70/83 (84.3%)	Report-defined operational coverage
Joint A and C	52.3%	60.5%	Report-defined combined outcome

All eligible executed probes passed in this run. The denominator is essential: P1 has 65 eligible executed probes and P2 has 76, rather than 83 probes for each system. The remaining recommendations are not functional failures; they were not in the eligible executed denominator. The package additionally records zero catalog-versus-probe failures, 13 cases where a probe passed despite missing catalog capability metadata, and 48 label-versus-operational discrepancies. These mismatches show why gold labels, metadata coverage, and execution probes should remain separate dimensions. A container

may execute a tested capability even when its catalog omits that capability, yet such underdeclared metadata remains a catalog quality issue.

The probes are bounded to single-process capability checks with workload memory capped at 1 GiB. They used digest-pinned approved images. GPU hardware execution was not claimed; CPU fallbacks were validated. Thus the observed 100% rates describe the executed eligible checks only. They do not certify all packages, numerical correctness, multi-process behavior, arbitrary inputs, or GPU execution.

4.3.3. E5: OCI layer accounting and catalog scale

The storage package uses a real registry manifest inspection collector to count compressed OCI manifest layer bytes for four approved images on linux/arm64. At the four-image prefix, the report records 11,392,935,522 logical descriptor bytes and 8,247,431,025 bytes across unique layer digests. The difference is 3,145,504,497 bytes, or 27.61% of the logical total. This is content-addressed manifest accounting; it is not a measurement of node filesystem allocation, unpacked image size, cache occupancy, or bytes transferred on every pull.

Table 4.5. Observed four-image compressed OCI layer-byte accounting

Quantity	Bytes
Logical ordered layer descriptor total	11,392,935,522
Unique content-addressed layer total	8,247,431,025
Logical-minus-unique difference	3,145,504,497
Difference as share of logical total	27.61%

The report also measures the marginal unique bytes introduced in its selected image order: 574,631,877 for minimal-python, 710,803,665 for scipy-data-science, 3,657,420,085 for pytorch-deep-learning, and 3,304,575,398 for tensorflow-deep-learning. The ordering and platform matter because the prefix total accumulates each image in that sequence and compares exact shared digests. The result is evidence of layer reuse in this

four-image catalog snapshot. It does not measure disk savings already realized on a node and should not be extrapolated linearly to larger catalogs.

The configured eight- and sixteen-image scales were not executed because only four approved images were available. The package includes a separate recommendation scalability row for the four-image case (83 reconstructed development cases, P2 acceptable and preferred accuracy 87.27%, Recall@5 77.73%, mean latency 2.23 ms). This result uses the reconstructed dataset and package-specific metric definitions; it is not interchangeable with the E1 primary report or the E5 functional report. The unexecuted larger scales have no observed recommendation or storage values.

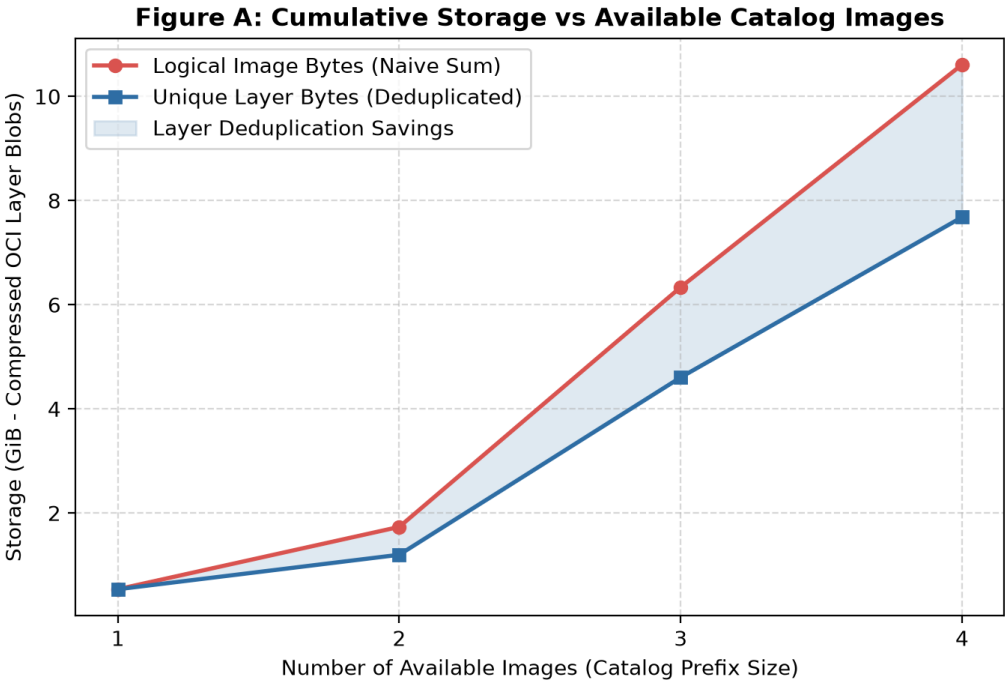


Figure 4.2. Logical and unique compressed OCI layer-byte totals for the observed four-image catalog prefix

4.4. Synthesis, limits, and implications

4.4.1. E3 and human outcomes

E3 is designed to compare B0 with P2 in a controlled, counterbalanced within-participant study. Participants would complete comparable environment-selection tasks in both conditions. The protocol records task correctness or acceptability, completion time, confidence, perceived workload, and usability. Task order should be counterbalanced to limit learning effects. The analysis should use participant-level outcomes and account for task and order, with all participant identifiers removed from published data.

A fair comparison requires equivalent task information, the same approved catalog, consistent onboarding, and a predefined scoring rubric. P2 should not receive hidden task context that B0 does not receive. Manual choices should be scored against preregistered acceptable sets, not a single subjective preferred profile when multiple options meet the task. B0 remains a manual choice and has no ranking metrics. The repository contains a smoke-test harness and protocol materials, but no participant responses. Consequently, no outcome in this thesis establishes reduced user time, improved human choice, confidence, lower workload, or usability.

4.4.2. P3 development gate and formal claim registry

P3 adds a grounded LLM reranker after P2 candidate generation. Its additional inference cost and output variability require evidence of sufficient headroom before retention. The development gate report records 11 eligible workload families, zero observed ranking-error families, and zero ranking-error attempts. The predefined minimum headroom condition was therefore not met and P3 was not retained. This is an observed development decision, not a confirmatory comparison or proof that every LLM reranker would fail to help.

The machine-generated consolidated thesis-results artifact remains INCOMPLETE and labels the registered confirmatory hypotheses NOT_EXECUTED. That registry answers whether its stricter confirmatory claim workflow completed. It

is distinct from whether local technical measurements were made. In the scope of the streamlined graduation-thesis guide, E1/E2 development metrics, E4 local trials, and E5 functional/storage observations are presented here as OBSERVED, with provenance, denominators, and limits. E3 remains genuinely unobserved because there are no participant data. Keeping these statuses separate permits the thesis to report what happened without claiming that the formal confirmatory hypotheses were accepted.

4.4.3. What the evidence supports

The current evidence supports several concrete statements: the P1/P2 offline evaluation generated the reported family-level estimates; the observed E4 package recorded 413 successes and 8 OOM events among 421 Pod trials; E5’s eligible probes all passed in their bounded execution set; and four approved image manifests exhibited the reported compressed-layer digest reuse. The P3 development gate did not retain its branch. Each statement is restricted to the source package, split, platform, denominator, and metric stated above.

The evidence does not support confirmatory P2 superiority, a universal OOM guarantee, participant benefit, full correctness of all images, GPU readiness, realized node-disk savings, or scaling to an eight- or sixteen-image catalog. The point estimates and local examples can guide follow-up engineering and evaluation, while the missing participant study and strict claim registry remain visible limitations. This interpretation neither downgrades measured data to “no evidence” nor converts bounded observations into claims beyond their scope.

For administrators, the practical implication is that a recommender should expose why a profile or image is proposed and preserve the user’s ability to inspect and change it. Capability metadata should be tested against actual environments, and resource envelopes should be derived from representative workload executions with explicit safety margins. For users, the preview makes the translation from workload description to environment choice inspectable. For researchers, the separate E1/E5 correctness rates and the E4

aggregate/per-family tension identify specific data-lineage questions to resolve in future reports.

Table 4.6. Evidence boundary for current thesis claims

Topic	Observed or supported statement	Not established by these data
E1/E2	Development estimates and aggregate robustness metrics are reported	Confirmatory superiority or general language robustness
E3	Protocol and harness are available; no participant data	Human benefit or usability outcomes
E4	421 local trials: 413 success, 8 OOM; 20 families	Universal OOM avoidance or general cluster efficiency
E5 functional	All 65 P1 and 76 P2 eligible executed probes passed	General image correctness or GPU execution
E5 storage	Four-image OCI descriptor and unique-byte accounting	Realized node storage or larger-scale projection
P3	Development gate observed zero ranking-error families and did not retain P3	Confirmatory comparison of P3 against P2

The overall result is a research prototype with a working recommendation path and several valuable, bounded empirical packages. The results are sufficiently concrete to inform the design discussion, but their denominators and provenance remain part of the finding itself. A future evaluation can improve confidence by reconciling the cross-package E1/E5 label metrics, mapping each E4 trial to its per-family envelope, collecting human outcomes, and measuring image behavior and storage on the intended deployment platform. Those are specific extensions of the evidence, rather than reasons to omit the observations already recorded.

Chapter 5.

Conclusion

5.1. Conclusion

This thesis presented Intent-Spawner, a bounded pre-spawn decision-support system for selecting JupyterHub environments from an administrator-curated catalog. P2 is the main proposed method. It validates a structured representation of current workload intent, uses BM25 and dense retrieval to generate candidates, fuses ranks with RRF, applies hard constraints before preference ranking, resolves selected identifiers from trusted catalog data, and binds the previewed choice to an authenticated user's confirmation. This design preserves a clear policy boundary and keeps the platform's ordinary admission and scheduling mechanisms authoritative.

The work also specified Protocol-v5 as a set of distinct experiment streams. That separation matters because recommendation relevance, robustness to prompt variation, human experience, resource adequacy, functional image behavior, and storage accounting are not interchangeable outcomes. The current repository contains development and local-environment observations across several of these streams, but the consolidated analysis is incomplete. Human participants have not been observed, several resource and image claims require audit and context-specific interpretation, and confirmatory decisions remain NOT_EXECUTED. Therefore, the principal empirical conclusion is a statement about evidence status: the prototype and evaluation infrastructure exist, but the registered claims have not yet been confirmed.

The system integration is the thesis's primary contribution. It shows how recommendation techniques can be applied at a practical JupyterHub configuration boundary while constraining effects through trusted catalog resolution, deterministic

feasibility rules, preview, and user confirmation. It does not show that intent predicts actual peak demand, that P2 is statistically superior to P1, or that P2 improves participant outcomes over B0. The local E4/E5 observations are useful for refining protocols and exposing operational questions, but they do not justify a broad production claim.

5.2. Limitations

Several limitations remain. First, offline recommendation evidence is development-stage and the current analysis package has metric and provenance reconciliation work outstanding. Second, the independent semantic sample is based on workload families, and 18 development families cannot support broad conclusions across all notebook domains. Third, E3 has no participant data, so the value of recommendations to real users is unknown. Fourth, E4's OrbStack environment differs from a representative multi-node cluster and must be reviewed for measurement-source, plan, and execution scope. Fifth, E5 tests only a small approved catalog, a particular platform, and bounded functional probes. Manifest layer bytes do not equal node disk use. Finally, a prototype preview-state design needs shared durable storage before it can be treated as suitable for multi-replica production deployment.

The feature scope is also intentionally narrow. P2 does not inspect live cluster capacity, schedule a Pod, guarantee successful admission, or replace runtime resource controllers. The dynamic-sizing component remains bounded prototype work. The reprovisioning workflow stops and recreates a server and cannot preserve in-memory execution state. P3 is not retained. These boundaries should remain visible in any abstract, defense presentation, or future system description.

5.2.1. Implications for administrators and users

For an administrator, the prototype suggests a workflow in which a candidate catalog is treated as versioned operational data rather than as an informal list of image names. Capability metadata should be testable, image references should be immutable, and

policy should be evaluated when a preview is created and again when it is applied. An administrator considering deployment should also decide how unsupported or ambiguous requests are communicated and whether any manual override remains available under local policy.

For a user, the intended value is less translation work. A user can express the task in familiar language and inspect a suggestion together with resource and capability information. The user should still be able to spot an unsuitable candidate, revise the intent, and understand when no approved profile satisfies a requirement. This interaction can reduce friction only if recommendations are sufficiently relevant and the preview is understandable. Since human evaluation has not been completed, these statements describe design intent, not demonstrated user benefit.

For a research evaluator, the project illustrates the importance of separating algorithmic evidence from integration and operational evidence. A recommender can be implemented correctly and still have weak rankings. A relevant candidate can fail a functional probe. A working Pod can still consume more resources than necessary. A successful task can still be harder for users to complete. The evaluation streams therefore provide a more useful roadmap than a single all-purpose score.

5.3. Prioritized future work

The next work should address evidence integrity before expanding the method. The first priority is to resolve the Protocol-v5 analysis package: reconcile E1 and E5 metric definitions, trace every derived value to raw observations, review dirty-tree provenance, and produce an eligible evidence-selection report. The second priority is to run frozen E1/E2 confirmation on sealed family-level cases. No implementation or configuration tuning should use those final cases.

The third priority is E3. The study should complete ethics and consent preparation, recruit an appropriate participant sample, counterbalance conditions, preserve privacy, and analyze participant-level outcomes using the registered plan. The fourth priority

is E4: close the plan-versus-observation audit, declare the eligible cgroup/Kubernetes resource source and conditions, and execute a protocol-conformant study on the intended cluster class. The fifth priority is E5 validation at the intended platform and catalog scope. Where eight- or sixteen-image catalogs do not exist, those cells should remain explicitly `NOT_EXECUTED` until suitable approved images are available.

Only after these work packages should development focus on improving top-one ranking, expanding catalog metadata, or studying optional dynamic sizing. P2 changes should be evaluated on development data and refrozen before any new confirmation. Multi-node, multi-tenant evaluation should examine quota contention, registry pulls, scheduling variability, policy drift, and failure recovery. Productionization should move preview state to a shared durable store and test atomic token consumption across replicas. Any future use of behavioral history requires a separate consent, retention, user-splitting, and privacy threat model.

5.3.1. Final thesis position

Intent-Spawner is best understood as an auditable, human-confirmed recommender for a bounded set of notebook environments. Its strongest demonstrated result is the concrete integration of intent parsing, retrieval, constraints, trusted selection, and pre-spawn application, together with an evaluation protocol that makes evidence boundaries explicit. The empirical record currently supports descriptive development and local observations only. Confirmatory comparisons and general deployment claims remain open work. Maintaining this distinction is necessary for an accurate final thesis and provides a clear path from the current prototype to defensible future conclusions.

References

- [1] Omid Alipourfard, Hongqiang Harry Liu, Jianshu Chen, Shivaram Venkataraman, Minlan Yu, and Ming Zhang. CherryPick: Adaptively unearthing the best cloud configurations for big data analytics. In *Proceedings of the 14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, pages 469–482, 2017.
- [2] Ali Anwar, Mohamed Mohamed, Vasily Tarasov, Michael Little, Lukas Rupprecht, Yue Cheng, Nannan Zhao, Dimitrios Skourtis, Amit S. Warke, He He, and Ali R. Butt. Improving Docker registry design based on production workload analysis. In *Proceedings of the 2018 USENIX Annual Technical Conference (USENIX ATC 18)*, pages 265–278, 2018.
- [3] John Brooke. SUS: A ‘quick and dirty’ usability scale. In *Usability Evaluation in Industry*, pages 189–194. Taylor & Francis, 1996.
- [4] Alexander Clemm, Laurent Ciavaglia, Leeyoung Lisheng, Kiran Makhijani, and Jeff Jefferts. Intent-based networking - concepts and definitions. RFC 9315, IETF, 2022.
- [5] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR 09)*, pages 758–759, 2009.
- [6] Eli Cortez, Anand Bonde, Alexandre Muzio, Mark Russinovich, Yang Qi, and Ricardo Bianchini. Resource Central: Understanding and predicting workloads for improved resource management in large cloud platforms. In *Proceedings of the 26th ACM Symposium on Operating Systems Principles (SOSP 17)*, pages 153–167, 2017.
- [7] Docker Documentation. About storage drivers. <https://docs.docker.com/engine/storage/drivers/>, 2026. Accessed: Sep. 2026.
- [8] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman

& Hall, 1993.

- [9] Alexander Felfernig, Gerhard Friedrich, Dietmar Jannach, and Markus Zanker. Constraint-based recommender systems. In *Recommender Systems Handbook*, pages 161–190. Springer, 2011.
- [10] Tyler Harter, Brandon Salmon, Rose Liu, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. Slacker: Fast distribution with lazy Docker containers. In *Proceedings of the 14th USENIX Conference on File and Storage Technologies (FAST 16)*, pages 181–195, 2016.
- [11] Jonathan L. Herlocker, Joseph A. Konstan, Loren G. Terveen, and John T. Riedl. Evaluating collaborative filtering recommender systems. *ACM Transactions on Information Systems*, 22(1):5–53, 2004.
- [12] Sture Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.
- [13] Anthony Jameson, Martijn C. Willemsen, Alexander Felfernig, Marco de Gemmis, Pasquale Lops, Giovanni Semeraro, and Li Chen. Human decision making and recommender systems. In *Recommender Systems Handbook*, pages 611–648. Springer, 2nd edition, 2015.
- [14] Kalervo Järvelin and Jaana Kekäläinen. Cumulated gain-based evaluation of IR techniques. *ACM Transactions on Information Systems*, 20(4):422–446, 2002.
- [15] JupyterHub Contributors. KubeSpawner documentation. <https://jupyterhub-kubespawner.readthedocs.io/>, 2026. Accessed: Sep. 2026.
- [16] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP 2020)*, pages 6769–6781, 2020.
- [17] Thomas Kluyver, Benjamin Ragan-Kelley, Fernando Pérez, Brian Granger, Matthias

- Bussonnier, Jonathan Frederic, Kyle Kelley, Jessica Hamrick, Jason Grout, Sylvain Corlay, Paul Ivanov, Damián Avila, Safia Abdalla, and Carol Willing. Jupyter notebooks—a publishing format for reproducible computational workflows. In *Positioning and Power in Academic Publishing: Players, Agents and Agendas*, pages 87–90. IOS Press, 2016.
- [18] Bart P. Knijnenburg, Martijn C. Willemsen, Zeno Gantner, Hakan Soncu, and Chris Newell. Explaining the user experience of recommender systems. *User Modeling and User-Adapted Interaction*, 22(4–5):441–504, 2012.
- [19] Kubernetes Authors. Kubernetes scheduler. <https://kubernetes.io/docs/concepts/scheduling-eviction/kube-scheduler/>, 2026. Accessed: Sep. 2026.
- [20] Kubernetes Authors. Pod quality of service classes. <https://kubernetes.io/docs/concepts/workloads/pods/pod-qos/>, 2026. Accessed: Sep. 2026.
- [21] Kubernetes Authors. Resource management for pods and containers. <https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/>, 2026. Accessed: Sep. 2026.
- [22] Kubernetes Authors. Resource quotas. <https://kubernetes.io/docs/concepts/policy/resource-quotas/>, 2026. Accessed: Sep. 2026.
- [23] Kubernetes Authors. Schedule GPUs. <https://kubernetes.io/docs/tasks/manage-gpus/scheduling-gpus/>, 2026. Accessed: Sep. 2026.
- [24] Kubernetes Autoscaling SIG. Vertical pod autoscaler. <https://github.com/kubernetes/autoscaler/tree/master/vertical-pod-autoscaler>, 2026. Accessed: Sep. 2026.
- [25] Aris Leivadeas and Matthias Falkner. A survey on intent-based networking. *IEEE Communications Surveys & Tutorials*, 25(1):625–655, 2023.
- [26] Xi Victoria Lin, Chenglong Wang, Luke Zettlemoyer, and Michael D. Ernst. NL2Bash: A corpus and semantic clips parser for natural language interface to

- the Linux operating system. In *Proceedings of the 11th International Conference on Language Resources and Evaluation (LREC 18)*, pages 3502–3509, 2018.
- [27] Yi Luan, Jacob Eisenstein, Kristina Toutanova, and Michael Collins. Sparse, dense, and attentional representations for text retrieval. *Transactions of the Association for Computational Linguistics*, 9:329–345, 2021.
- [28] Open Container Initiative. OCI image format specification. <https://github.com/opencontainers/image-spec>, 2026. Accessed: Sep. 2026.
- [29] Raja Parasuraman, Thomas B. Sheridan, and Christopher D. Wickens. A model for types and levels of human interaction with automation. *IEEE Transactions on Systems, Man, and Cybernetics - Part A: Systems and Humans*, 30(3):286–297, 2000.
- [30] Project Jupyter. JupyterHub documentation. <https://jupyterhub.readthedocs.io/>, 2026. Accessed: Sep. 2026.
- [31] Pearl Pu, Li Chen, and Rong Hu. A user-centric evaluation framework for recommender systems. In *Proceedings of the 5th ACM Conference on Recommender Systems (RecSys 11)*, pages 157–164, 2011.
- [32] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP 2019)*, pages 3982–3992, 2019.
- [33] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.
- [34] Michał Rządca, Paweł Findeisen, Jacek Świątek, Przemysław Zych, Przemysław Bialik, Marta Siedlecka, Jacek Wójcik, Dominik Krysiuk, Zuzanna Skowron, Ricardo Koller, and John Wilkes. Autopilot: Workload autoscaling at Google. In *Proceedings of the 15th European Conference on Computer Systems (EuroSys 20)*, pages 1–16, 2020.

- [35] Jeff Sauro and Joseph S. Dumas. Comparison of three one-question, post-task usability questionnaires. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI 09)*, pages 1599–1608, 2009.
- [36] Shivaram Venkataraman, Zongheng Yang, Michael Franklin, Benjamin Recht, and Ion Stoica. Ernest: Efficient performance prediction for large-scale advanced analytics. In *Proceedings of the 13th USENIX Symposium on Networked Systems Design and Implementation (NSDI 16)*, pages 363–378, 2016.
- [37] Qin Wu, Alexander Clemm, Mohamed Boucadair, Christian Jacquenet, and Chongfeng Xie. Intent-driven cloud resource design. In *Proceedings of the 2021 IEEE Cloud Summit*, pages 141–146, 2021.
- [38] Zero to JupyterHub Contributors. Zero to JupyterHub with Kubernetes. <https://z2jh.jupyter.org/>, 2026. Accessed: Sep. 2026.